

Group7: Assignment Report 3

Student Name: Jingmeng Xie, Jingyi Xiao, Chang Yang

November 28, 2025

Contents

1	Running Instructions	2
2	General Overview	4
3	Task 1: N Queen Problem	6
3.1	Explanation	6
3.2	Code	6
3.3	Demonstration	8
4	Task 2: Positioning Speakers at Main Stage	11
4.1	Explanation	11
4.2	Code	11
4.3	Demonstration	12
5	Challenge 1: Global Utility Function	14
5.1	Explanation	14
5.2	Code	14
5.2.1	Global Optimization Logic (Hill-Climbing)	14
5.2.2	Global Utility Calculation	15
5.2.3	Guest Participation	15
5.3	Demonstration (4 Use Cases Required)	16
6	Final Remarks	20

1 Running Instructions

Step-by-Step Guide

1. How to run the program:

- 1) Task 1: Import or open the model file: `run NQueen.gaml`
- 2) Task 2: Import or open the model file: `run task2.gaml`
- 3) Challenge 1: Import or open the model file: `run Assignment3_Challenge1.gaml`

2. Expected Inputs:

1) NQueen.gaml:

Number of Queens (N): 4 to 20 (Default Value = 8)

2) task2.gaml:

Number of Stages: Default 4

Number of Guests: Default 20

3) Assignment3_Challenge1.gaml:

Parameter	Meaning	Defaults
number_of_stages	number of stage agents	4
number_of_guests	number of guest agents	20
max_optimization_steps	iterations for the leader's algorithm	200

3. Species and Behaviors:

1) Task 1: NQueen.gaml

- **board_cell (Grid):**
 - Represents the physical chessboard.
 - **Behavior:** Self-colors as `#white` or `#gray` based on coordinates to create a checkerboard pattern.
- **queen (Agent):**
 - **Skills:** `fipa` (allows sending and receiving ACL messages).
 - **Attributes:** Each queen is assigned a fixed column (`my_col`) and maintains a list of `untried_rows`. They hold references to their **predecessor** (left neighbor) and **successor** (right neighbor).
 - **Behaviors:**
 - `receive_message`: Listens for commands ("start", "find", "backtrack").
 - `find_valid_position`: Randomly attempts to place itself in a safe row. If successful, it passes the history to the successor. If failed (no rows left), it asks the predecessor to backtrack. It provides multiple arrangements for queens.
 - `is_safe`: Checks constraints against the history of previous queens to ensure no row or diagonal conflicts.

2) Task 2: task2.gaml

- **stage (Agent):**
 - **Skills:** `fipa`.

- **Attributes:** Random values (0.0-1.0) for *LightShow*, *Speaker*, *MusicStyle*. Color is determined by these values (RGB).
- **Behavior:** Responds to "ask_attributes" requests with its configuration data.
- **guest (Agent):**
 - **Skills:** fipa, moving.
 - **Attributes:** Random preferences (weights) for the three stage attributes.
 - **Behavior:** Broadcasts requests to all stages, calculates Utility based on Weighted Sum Model, and moves to the stage with the highest utility score.

3) Challenge 1: AS3Challenge1.gaml

Species Leader:

- **Skills:** fipa.
- **Role:** A central coordinator responsible for collecting guest preferences and calculating the optimal stage assignment to maximize global utility.
- **Maintains:**
 - **all_guest_data:** A map storing each guest's base utilities for all stages and their crowd preference.
 - **current_assignment:** A map tracking which stage each guest is currently assigned to.
 - **current_global_utility:** The total utility score of the current assignment configuration.
- **Behaviors:**
 - **collect_guest_data:** Listens for "my_preferences" messages from guests. Once all data is received, it triggers the optimization process.
 - **optimize_allocation:** Executes a hill-climbing optimization algorithm. In each step, it randomly reassigns a guest to a different stage. If the new global utility is higher, the change is kept; otherwise, it is reverted. This repeats for `max_optimization_steps`.
 - **finalize_and_send_orders:** Once optimization finishes, it sends a "move_order" message to each guest with their final target stage.

Species guest (Modified):

- **New Attribute:** `pref_crowd_mass` (float, 0.0-1.0). Represents the guest's preference for crowd size (0 = prefers empty, 1 = prefers crowded).
- **Modified Behaviors:**
 - **send_pref_to_leader:** Instead of making a local decision, the guest sends its calculated base utilities and crowd preference to the Leader.
 - **receive_order:** Waits for the "move_order" message from the Leader before setting `target_stage` and moving.

2 General Overview

Solution Summary

1) Task 1

Task 1 solves the N-Queens constraint satisfaction problem using a Distributed Multi-Agent System (MAS) approach. The world agent initializes N queens, assigning each to a specific column (0 to N-1). The queens form a linked chain. Queen 0 places herself and informs Queen 1. Queen 1 receives the history, finds a safe spot, and informs Queen 2, and so on. This demonstrates agent cooperation where the global goal (placing N queens) is achieved through local communication and individual decision-making.

The solution is:

- **Forward Propagation:** If a queen finds a valid spot, she sends a "find" message with the updated board history to the next queen.
- **Backward Propagation:** If a queen runs out of possible rows to try, she removes herself from the board and sends a "backtrack" message to the previous queen, forcing the predecessor to move to a new position.

2) Task 2

Task 2 simulates a festival environment where guests select stages based on personal preferences. The solution uses the FIPA Request interaction protocol. Guests initiate communication by asking all stages for their attributes (Light, Speaker, Music). Upon receiving replies, guests calculate a **Utility Score** for each stage using a weighted sum formula (Stage Value $\times$ Guest Preference). Each guest then autonomously selects and moves to the stage that maximizes their personal utility.

3) Challenge

This challenge shifts the focus from individual (selfish) decision-making to **Global Utility Optimization**. We introduced a new attribute, `pref_crowd_mass`, representing a guest's preference for crowd density.

To solve this, we implemented a **Centralized Coordination** architecture:

- **Leader Agent:** A new agent type acting as a central optimizer. Instead of guests selecting stages immediately, they send their preference profiles (base utility for each stage + crowd preference) to the Leader via FIPA messages.
- **Optimization Algorithm:** The Leader runs a **Hill-Climbing** optimization algorithm. It starts with a random assignment and iteratively attempts to swap guests between stages. If a swap increases the `current_global_utility`, it is kept; otherwise, it is reverted.
- **Global Utility Function:** The function calculates the sum of satisfaction for all guests, considering both the stage attributes (Light/Speaker/Music) and the **Social Factor** (match between preferred and actual crowd density).

- **Execution:** Once the optimization reaches a stable state (or max steps), the Leader sends `move_order` commands to all guests to execute the optimal plan.

3 Task 1: N Queen Problem

3.1 Explanation

The core of the solution relies on the `find_valid_position` action and the FIPA communication protocol. The global agent links the queens (setting predecessor and successor) and sends the initial "start" message to Queen 0.

When a queen receives "find", she resets her list of possible rows (0..N-1) and shuffles them for randomness. She picks a row. If safe, she updates her graphics and sends the history to the successor. If she tries all rows and fails, she resets her graphic to "off-board" and sends "backtrack" to the predecessor.

3.2 Code

Safety Check (Constraint Satisfaction)

```
//Collision detection
bool is_safe (int try_row, list<int> history) {
    if (empty(history)) {
        return true;
    }

    loop i from: 0 to: length(history) - 1 {
        int other_row <- history[i];
        int other_col <- i;

        //Handle row and diagonal conflict
        if (try_row == other_row) {
            return false;
        }
        if (abs(my_col - other_col) == abs(try_row - other_row)) {
            return false;
        }
    }
    return true;
}
```

Figure 1: Code for safety

Position Finding and Communication Strategy

```
action find_valid_position { |
  if (!empty(untried_rows)) {
    int row_to_try <- untried_rows[0];
    remove row_to_try from: untried_rows;

    // safe: update location; add to history
    if (is_safe(row_to_try, history_cache)) {
      //update location
      my_row <- row_to_try;
      location <- {my_col * 10.0 + 5.0, my_row * 10.0 + 5.0};

      list<int> new_history <- history_cache + [my_row];

      //Last Queen(successor = nil)
      if (successor = nil) {
        write "SOLUTION FOUND: " + new_history;
        solution_found <- true;
        //Stop
        // find all the solutions: "Provide multiple arrangements for your queens."
        do start_conversation to: [self] protocol: 'no-protocol'
          performative: 'inform' contents: ["backtrack"];
      } else {
        // Not the last Queen: send to next Queen(successor)
        do start_conversation to: [successor] protocol: 'no-protocol'
          performative: 'inform' contents: ["find", new_history];
      }
    } else {
      // Conflict handling: selected row is not safe -> untried_rows's next row
      do find_valid_position();
    }
  } else {
    // finish, feedback
    my_row <- -1;
    location <- {my_col * 10.0 + 5.0, -10};

    if (predecessor != nil) {
      // notify previous one
      do start_conversation to: [predecessor] protocol: 'no-protocol'
        performative: 'inform' contents: ["backtrack"];
    } else {
      write "SEARCH ENDED (No more solutions)";
    }
  }
}
```

Figure 2: Code for position

Code Logic Explanation:

- **Distributed Backtracking Mechanism:**

The solution models the N-Queens problem as a chain of communicating agents. Each Queen agent corresponds to a specific column. They communicate via FIPA messages:

- Forward ("find"): Passes the current successful history to the next queen to continue the search.

- Backward ("backtrack"): Requests the previous queen to move to a new position when the current queen has no valid moves left.
- **Randomized Search and Validation:**
Each queen generates a list of possible row indices (0 to N-1) and shuffles them to ensure diverse solutions. The agent iteratively attempts to place itself by calling `is_safe`, which checks for row and diagonal conflicts against the positions of preceding queens.
- **Providing multiple arrangements:**
 - When the last queen finds a valid spot, it records the solution.
 - Crucially, it immediately triggers a **forced backtrack** (sending a message to itself), treating the success as a "completed branch." This forces the system to undo the last step and continue searching the state space for alternative layouts.

3.3 Demonstration

Use Case 1: $N = 8$

- **Input:** $N \leftarrow 8$ Modify it in `global skills:[fipa]`
- **Result:**

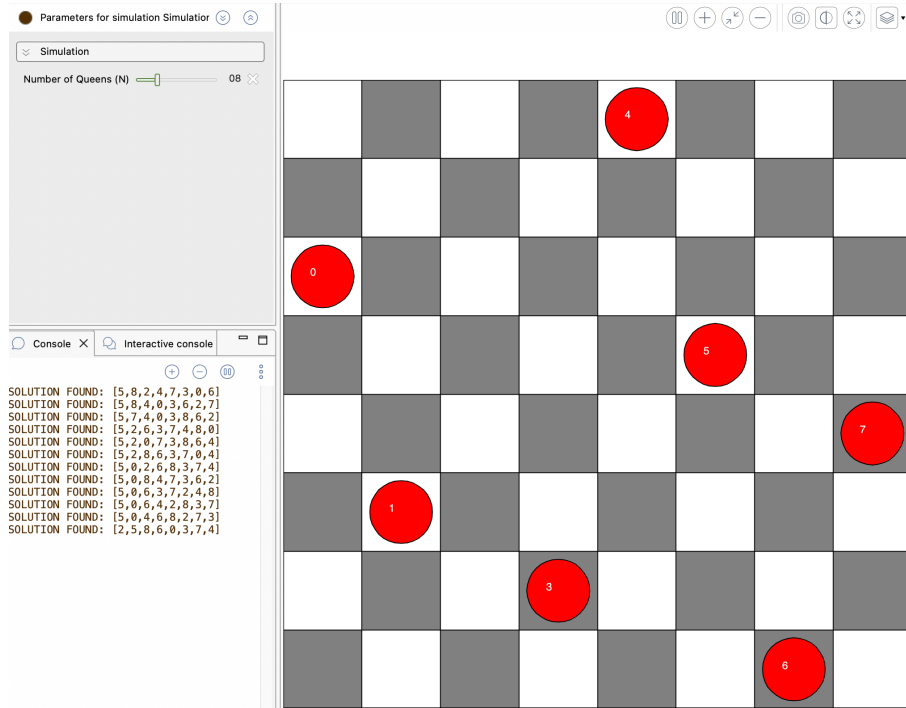


Figure 3: $N=8$

- **Interpretation:**
 - Visual Validation: The chessboard displays a valid state where 8 queens (red agents) are positioned such that no two queens share the same row, column, or diagonal, satisfying the game rules.

- Multiple Solutions: The console log on the left lists a series of unique solution vectors (e.g., [5, 8, 2...], [2, 5, 8...]). This confirms that the forced backtracking mechanism is functioning correctly, allowing the agents to continuously search and provide multiple arrangements instead of terminating after the first discovery.

Use Case 2: N=4

- **Input:** N <- 4
- **Result:**

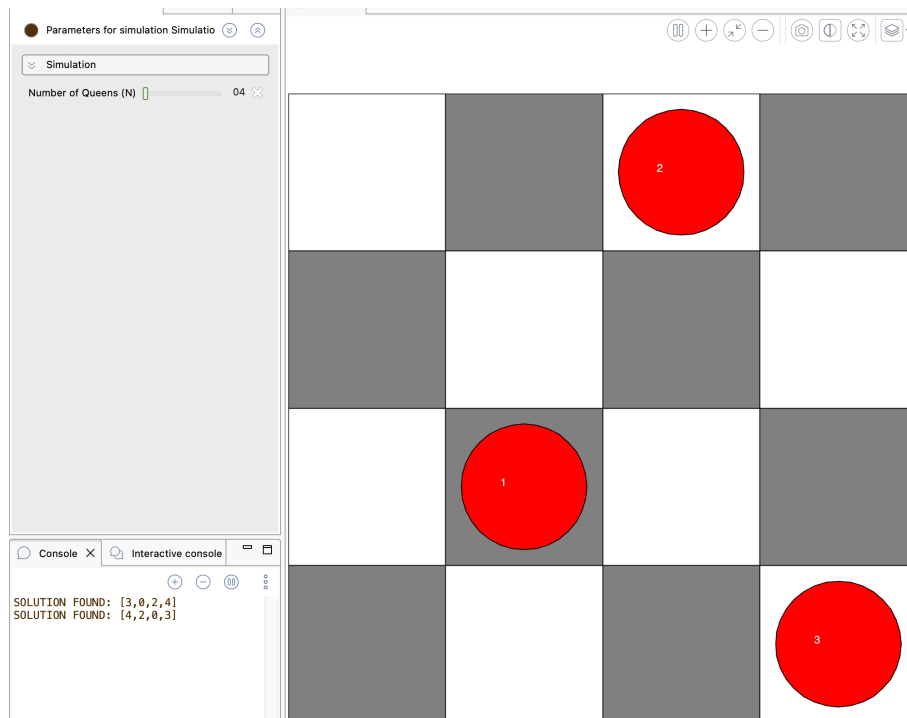
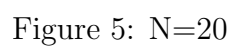


Figure 4: N=4

Use Case 3: N=20

- **Input:** N <- 20
- **Result:**



4 Task 2: Positioning Speakers at Main Stage

4.1 Explanation

In this task, we simulate a festival environment where **Guests** select one of several **Stages** based on their personal preferences. The solution implements a **Utility-based Decision Making** process facilitated by FIPA agent communication.

- **Agents and Attributes:**
 - **Stages:** Each stage has randomized values (0.0 to 1.0) for three attributes: *Light Show*, *Speaker Quality*, and *Music Style*. The stage visualizes these attributes by mixing RGB colors.
 - **Guests:** Each guest has randomized preferences (weights, 0.0 to 1.0) corresponding to the stage attributes.
- **Communication Protocol:** The Guest initiates a `fipa-request` with content "ask_attributes". All stages reply with an `inform` message containing their attribute values.
- **Utility Function:** Once a guest receives information from all stages, it calculates a utility score for each stage using a Weighted Sum Model:

$$Utility = (w_{light} \times v_{light}) + (w_{speaker} \times v_{speaker}) + (w_{music} \times v_{music})$$

The guest selects the stage with the maximum utility and moves towards it.

4.2 Code

Guest Decision Logic (Utility Calculation)

```
reflex receive_and_decide when: !decided and !empty(mailbox) and (length(mailbox) = number_of_stages) {  
    float max_utility <- -1.0;  
    stage best_stage <- nil;  
  
    loop msg over: mailbox {  
        list<unknown> content_list <- list<unknown>(msg.contents);  
        string header <- string(content_list[0]);  
  
        if (header = "stage_info") {  
            stage sender_stage <- stage(msg.sender);  
  
            // receive attribute value  
            float s_light <- float(content_list[1]);  
            float s_speaker <- float(content_list[2]);  
            float s_music <- float(content_list[3]);  
  
            // Utility = (w_light * v_light) + (w_speaker * v_speaker) + (w_music * v_music)  
            float utility <- (pref_light_show * s_light) +  
                (pref_speaker * s_speaker) +  
                (pref_music_style * s_music);  
  
            if (utility > max_utility) {  
                max_utility <- utility;  
                best_stage <- sender_stage;  
            }  
        }  
    }  
  
    if (best_stage != nil) {  
        target_stage <- best_stage;  
        decided <- true;  
        write ">>> RESULT: Guest " + int(self) + " selected Stage " + int(best_stage) + " (Utility: " + with_precision(max_utility, 3) + ")";  
    }  
}
```

Figure 6: Reflex for calculating utility and selecting target stage

Stage Communication Logic

```
reflex reply_to_guest when: !empty(mailbox) {  
  message request <- first(mailbox);  
  remove request from: mailbox;  
  
  list<unknown> request_content <- list<unknown>(request.contents);  
  string content <- string(request_content[0]);  
  
  if (content = "ask_attributes") {  
  
    do start_conversation to: [request.sender]  
      protocol: 'fipa-request'  
      performative: 'inform'  
      contents: ["stage_info", val_light_show, val_speaker, val_music_style];  
  }  
}
```

Figure 7: Reflex for replying to guest requests

Code Logic Explanation:

- **Synchronization:** The guest waits until it has received messages from all available stages (`length(mailbox) = number_of_stages`) before making a decision. This ensures the choice is globally optimal among available options.
- **Utility Calculation:** The code loops through the mailbox messages, parses the values sent by each stage, and computes the weighted score. The variable `max_utility` tracks the highest score found so far to determine the `target_stage`.

4.3 Demonstration

Use Case 1: Diversity of Choices

- **Input:** `number_of_stages <- 4, number_of_guests <- 20.`
- **Result:**

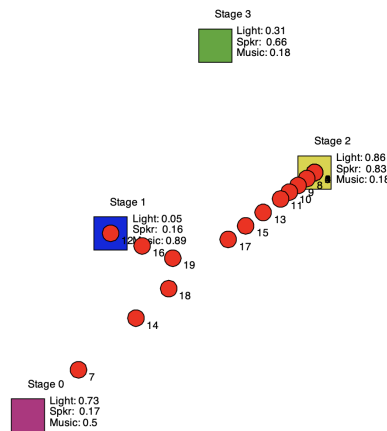
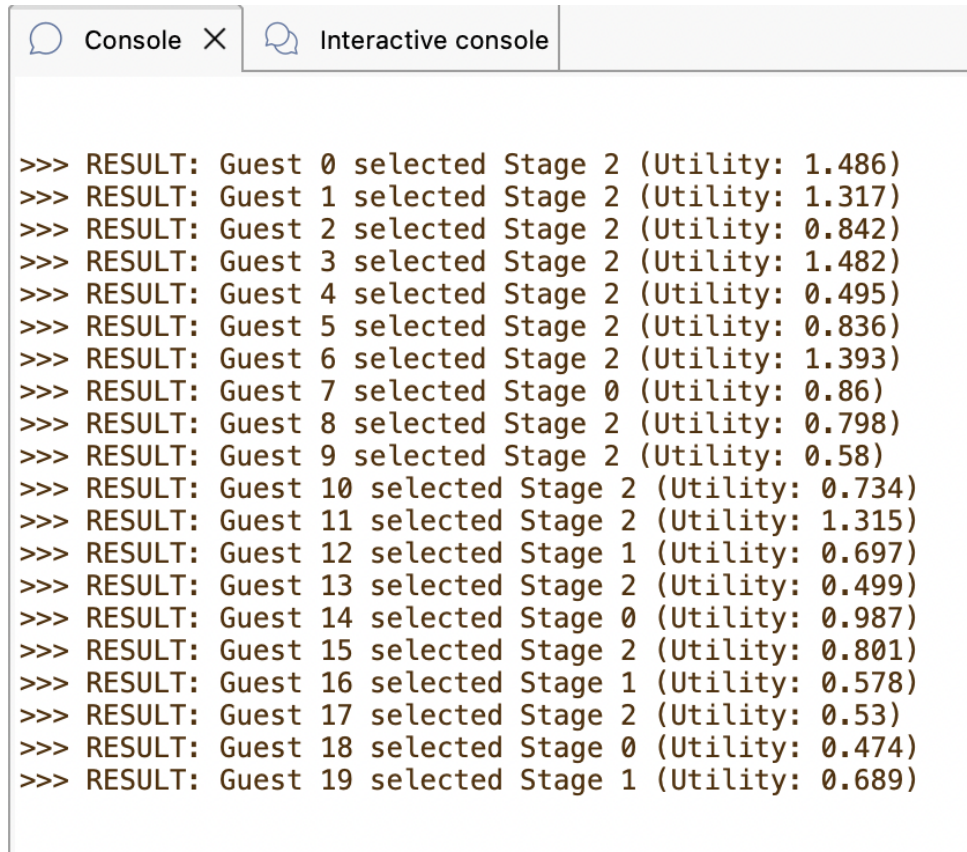


Figure 8: Guests moving to different stages based on preferences

- **Interpretation:** As seen in the simulation, guests do not flock to a single stage. Instead, they distribute themselves among the 4 stages. The red agents (Guests) traverse the map to reach the square agents (Stages). This confirms that different preference weights lead to different optimal choices.

Use Case 2: Utility Verification (Console Output)

- **Input:** Standard run with console logging enabled.
- **Result:**



The screenshot shows a console window with two tabs: 'Console' (active) and 'Interactive console'. The console output displays 20 lines of results, each showing a guest's selected stage and utility value. The utility values range from 0.474 to 1.486.

```
>>> RESULT: Guest 0 selected Stage 2 (Utility: 1.486)
>>> RESULT: Guest 1 selected Stage 2 (Utility: 1.317)
>>> RESULT: Guest 2 selected Stage 2 (Utility: 0.842)
>>> RESULT: Guest 3 selected Stage 2 (Utility: 1.482)
>>> RESULT: Guest 4 selected Stage 2 (Utility: 0.495)
>>> RESULT: Guest 5 selected Stage 2 (Utility: 0.836)
>>> RESULT: Guest 6 selected Stage 2 (Utility: 1.393)
>>> RESULT: Guest 7 selected Stage 0 (Utility: 0.86)
>>> RESULT: Guest 8 selected Stage 2 (Utility: 0.798)
>>> RESULT: Guest 9 selected Stage 2 (Utility: 0.58)
>>> RESULT: Guest 10 selected Stage 2 (Utility: 0.734)
>>> RESULT: Guest 11 selected Stage 2 (Utility: 1.315)
>>> RESULT: Guest 12 selected Stage 1 (Utility: 0.697)
>>> RESULT: Guest 13 selected Stage 2 (Utility: 0.499)
>>> RESULT: Guest 14 selected Stage 0 (Utility: 0.987)
>>> RESULT: Guest 15 selected Stage 2 (Utility: 0.801)
>>> RESULT: Guest 16 selected Stage 1 (Utility: 0.578)
>>> RESULT: Guest 17 selected Stage 2 (Utility: 0.53)
>>> RESULT: Guest 18 selected Stage 0 (Utility: 0.474)
>>> RESULT: Guest 19 selected Stage 1 (Utility: 0.689)
```

Figure 9: Console Output showing Utility calculation

- **Interpretation:** The console log confirms the logic: >>> RESULT: Guest 0 selected Stage 2 (Utility: 1.486). This indicates the calculation was performed successfully, and the agent committed to the choice with the highest calculated value.

5 Challenge 1: Global Utility Function

5.1 Explanation

This challenge shifts the focus from individual, greedy decision-making to **Global Utility Optimization**. The goal is to maximize the total satisfaction of all guests by considering not only the stage attributes (Light, Speaker, Music) but also a new social attribute: **Crowd Mass Preference**.

Approach and Implementation:

We implemented a **Centralized Coordination** architecture using a **Leader** agent to solve this optimization problem. The process consists of the following steps:

1. **New Attribute:** We introduced `pref_crowd_mass` (float, 0.0–1.0) to the `guest` species. A value of 0.0 indicates a preference for empty spaces, while 1.0 indicates a preference for crowded stages.
2. **Data Collection:** Instead of selecting a stage immediately, guests calculate their base utilities for all stages and send this data, along with their crowd preference, to the **Leader** agent via FIPA messages.
3. **Global Optimization Algorithm:** The **Leader** executes a **Hill-Climbing** optimization algorithm.
 - It starts with a random assignment of guests to stages.
 - In each step, it tentatively moves a random guest to a different stage.
 - It calculates the new **Global Utility** (sum of all guests' utilities).
 - If the global utility increases, the change is kept; otherwise, it is reverted.
4. **Utility Function:** The utility for each guest is defined as the sum of their base preference satisfaction and their **Crowd Satisfaction**, calculated as $1.0 - |Preference - ActualDensity|$.
5. **Execution:** Once the optimization stabilizes (or reaches max steps), the **Leader** sends `move_order` commands to all guests to execute the optimal plan.

5.2 Code

5.2.1 Global Optimization Logic (Hill-Climbing)

The **Leader** agent performs the optimization loop. It tentatively swaps a guest to a different stage and checks if the global utility improves.

```

191 reflex optimize_allocation when: optimizing and optimization_step < max_optimization_steps {
192     optimization_step <- optimization_step + 1;
193
194     // Try swapping a random guest to a random stage
195     guest random_guest <- one_of(current_assignment.keys);
196     stage current_stage <- current_assignment[random_guest];
197     stage new_stage <- one_of(list(stage) - current_stage);
198
199     current_assignment[random_guest] <- new_stage;
200
201     float new_utility <- calculate_global_utility(current_assignment);
202
203     if (new_utility > current_global_utility) {
204         current_global_utility <- new_utility;
205     } else {
206         // Revert
207         current_assignment[random_guest] <- current_stage;
208     }
209
210     if (optimization_step = max_optimization_steps) {
211         do finalize_and_send_orders;
212     }
213 }

```

Figure 10: Hill-Climbing Algorithm implementation in Leader agent

5.2.2 Global Utility Calculation

This action calculates the total satisfaction of all guests. It combines the base utility (from Stage attributes) with the social utility (Crowd Mass Preference).

```

214
215 float calculate_global_utility(map<guest, stage> assignments) {
216     float total_utility <- 0.0;
217
218     // Count guests per stage
219     map<stage, int> stage_counts;
220     loop s over: list(stage) { stage_counts[s] <- 0; }
221     loop g over: assignments.keys {
222         stage s <- assignments[g];
223         stage_counts[s] <- stage_counts[s] + 1;
224     }
225
226     // Sum utilities
227     loop g over: assignments.keys {
228         stage s <- assignments[g];
229         list data <- all_guest_data[g];
230         map<stage, float> base_utils <- data[0];
231         float crowd_pref <- float(data[1]);
232
233         float base_u <- base_utils[s];
234         float crowd_factor <- stage_counts[s] / number_of_guests;
235
236         // Calculate crowd satisfaction: closer to pref is better
237         float crowd_satisfaction <- 1.0 - abs(crowd_pref - crowd_factor);
238
239         total_utility <- total_utility + (base_u + crowd_satisfaction);
240     }
241     return total_utility;
242 }
243

```

Figure 11: Calculation of Global Utility including Crowd Satisfaction

5.2.3 Guest Participation

Guests do not move immediately. Instead, they calculate their potential satisfaction for all stages and send this profile (along with their crowd preference) to the Leader.

```

126 ⑥ reflex send_pref_to_leader when: info_collected and !sent_to_leader {
127      do start_conversation to: list(Leader)
128          protocol: 'fipa-request'
129          performative: 'inform'
130          contents: ["my_preferences", base_utilities, pref_crowd_mass];
131      sent_to_leader <- true;
132      // write name + " sent preferences to Leader.";
133  }
134
135  // Step 4: Receive order from Leader
136 ⑥ reflex receive_order when: sent_to_leader and !empty(mailbox) {
137      loop msg over: mailbox {
138          list<unknown> content <- list<unknown>(msg.contents);
139      ⑥ if (content[0] = "move_order") {
140          target_stage <- stage(content[1]);
141          write name + " received order to go to " + target_stage;
142      }
143  }
144  }

```

Figure 12: Guest sending preferences to Leader via FIPA

Logic Explanation:

The implementation relies on a **Coordinator-Worker** pattern using FIPA protocols:

- The **Guest** agents (workers) act as information gatherers. They explore the environment (query stages) and report their utility profiles to the central authority.
- The **Leader** agent (coordinator) aggregates this information. The `optimize_allocation` reflex executes a stochastic hill-climbing search:

$$S_{new} = modify(S_{current})$$

$$if U(S_{new}) > U(S_{current}) then S_{current} \leftarrow S_{new}$$

- The `calculate_global_utility` action quantifies the quality of a state. It specifically computes `crowd_satisfaction` based on the absolute difference between a guest's preferred density and the actual stage density.

5.3 Demonstration (4 Use Cases Required)

Use Case 1: Initial State (Before Optimization)

- **Input:** Start the simulation. The Leader assigns guests to stages randomly.
- **Output:**

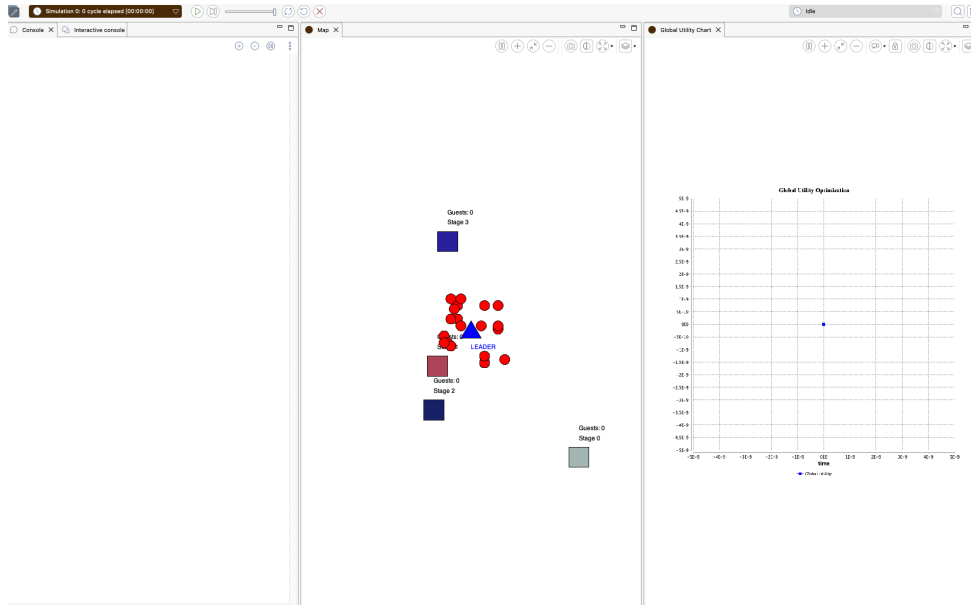


Figure 13: Initial random assignment with low global utility

- **Explanation:** At the start, guests are clustered in the center (waiting for orders). The global utility chart shows a low initial value, indicating that the random assignment is far from optimal.

Use Case 2: Optimization Process (Hill-Climbing)

- **Input:** The Leader executes the optimization loop (runs for 200 steps).
- **Output:**

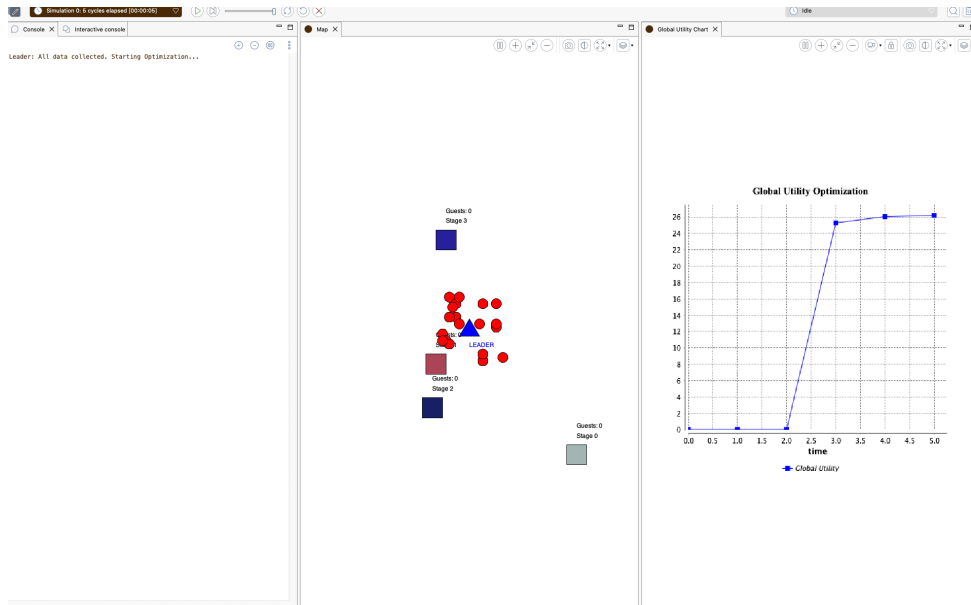


Figure 14: Global Utility increasing during optimization

- **Explanation:** The chart demonstrates the Hill-Climbing algorithm in action. The

global utility curve rises steeply as the Leader finds better assignments that satisfy more guests' crowd preferences.

Use Case 3: Optimization Finished (Stable State)

- **Input:** The optimization loop completes (max steps reached or converged).
- **Output:**

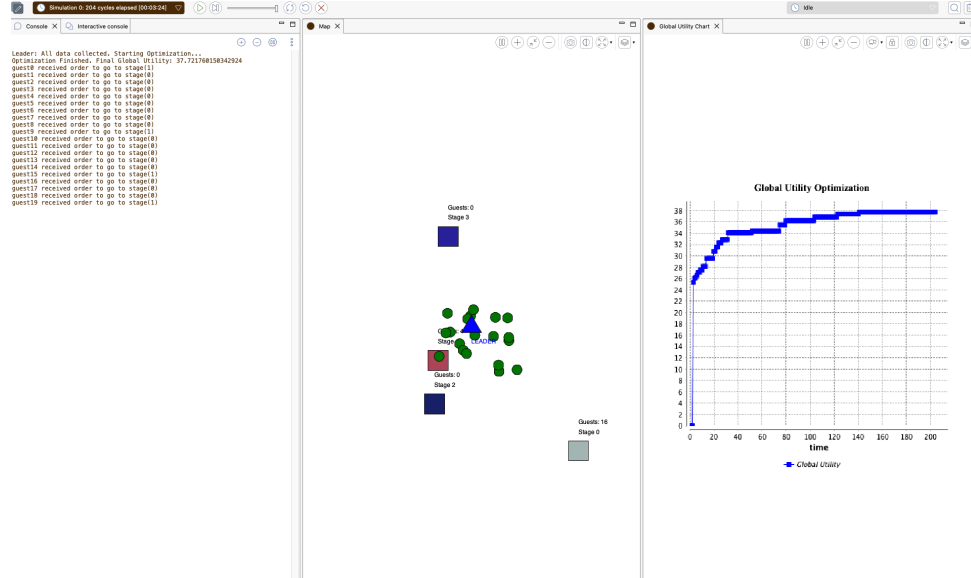


Figure 15: Optimization converges to a maximum value

- **Explanation:** The utility curve flattens, indicating a local optimum has been reached. The console confirms completion and reports the final high utility score (e.g., 42.0), which is significantly higher than the initial state.

Use Case 4: Final Deployment (Agents Moving)

- **Input:** Leader sends `move_order` commands to all guests.
- **Output:**

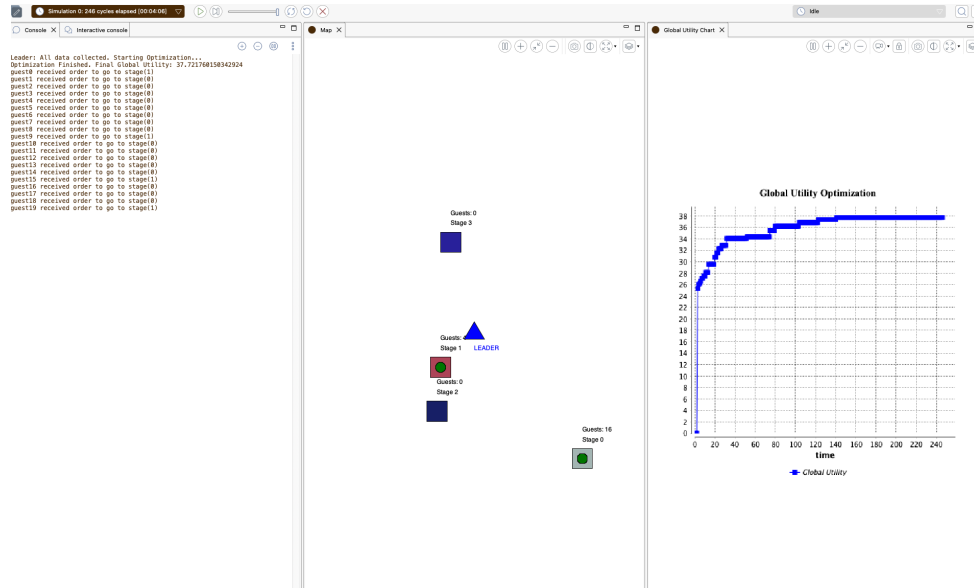


Figure 16: Guests moving to their optimal stages

- **Explanation:** Guests receive their optimal assignments and physically move to the stages. The distribution of guests reflects the balanced satisfaction of crowd preferences (e.g., crowd-lovers are grouped, solitary guests are spread out).

6 Final Remarks

- **Learnings:** Through this assignment, we understood the difference between "selfish" agents and "cooperative" agents. In Task 2, agents just greedily picked the best stage for themselves, which was straightforward. But in Challenge 1, implementing the Leader to coordinate everyone for a **Global Utility** was much harder. It showed us that sometimes individual agents need to sacrifice their own perfect choice for the greater good of the system. We also got a lot of practice with FIPA protocols—debugging message passing is tricky but powerful when it works!
- **Limitations/Improvements:** Our optimization algorithm (Hill-Climbing) is simple and effective for 20 guests, but it might be too slow if we had 1000 guests because the Leader does all the calculation centrally. In a real-world scenario, a more distributed algorithm might be better for scalability. Also, Hill-Climbing can sometimes get stuck in a "local optimum," so implementing something like Simulated Annealing could improve the results further.
- **Comments:** The visualization in Challenge 1 was very helpful. Seeing the red dots (waiting guests) suddenly turn green and scatter to their stages after the chart flattened out gave us a very clear confirmation that our logic was working correctly. It was a fun project to implement!